

2.5 — Select and Apply Built-in Tools Effectively

Concept

Claude Code provides five built-in tools that cover the full lifecycle of working with a codebase: **Grep**, **Glob**, **Read**, **Write**, and **Edit**. Selecting the right tool for each step is not a minor optimization — it directly determines whether the agent can complete its task within the context window and without introducing errors.

Domain mental model: Tool descriptions are the LLM's only guide to what a tool does and when to use it. Design them like API docs — precise, prescriptive, and explicit about preconditions and failure modes.

The Five Built-in Tools

Tool	Purpose	Input	When to Use
Grep	Search file <i>contents</i> by regex pattern	Pattern, optional path/glob	You know <i>what</i> you're looking for but not <i>where</i> it is
Glob	Match file <i>paths</i> by pattern	Glob pattern, optional directory	You know the naming convention or directory structure
Read	View the contents of a specific file	Absolute path, optional line range	You know the exact file and need to inspect it
Write	Create or completely overwrite a file	Absolute path, full content	Creating a new file; full rewrites after reading
Edit	Replace a specific string within an existing file	Absolute path, old string, new string	Targeted modifications to an existing file

Incremental Codebase Understanding

The most common mistake agents make is attempting to read the entire codebase before acting. This exhausts the context window and frequently fails. The correct approach is a progressive narrowing strategy:

1. **Broad discovery with Glob** — Identify which files exist and where they live. Use patterns like `**/*.ts` or `src/**/*.config.*` to orient yourself without reading any content.
2. **Content search with Grep** — Locate the specific function, variable, or pattern you need. Use regex to find the exact symbol across matched files.
3. **Targeted inspection with Read** — Only once you know which file is relevant, read it. Use the `offset` and `limit` parameters to read large files in sections rather than all at once.
4. **Precise modification with Edit** — Make the smallest possible change. Never use Write to modify an existing file unless the change spans the entire file.

This four-step funnel keeps context usage proportional to the task.

Code Example

Scenario: You need to update the error message in an authentication handler, but you don't know which file it lives in.

```
# Step 1 – Glob to orient
Glob: "src/**/*.ts"
→ Returns 47 file paths. Now you know the structure.

# Step 2 – Grep to locate
Grep: pattern="unauthorized", path="src/", type="ts"
→ Returns: src/auth/middleware.ts, src/api/handlers/login.ts

# Step 3 – Read to inspect
Read: file_path="/project/src/auth/middleware.ts", offset=45, limit=30
→ Line 52: throw new Error("unauthorized access")

# Step 4 – Edit to modify
Edit:
file_path="/project/src/auth/middleware.ts"
old_string='throw new Error("unauthorized access")'
new_string='throw new Error("Access denied. Please check your credentials.")'
```

Four tool calls. Zero wasted context. No ambiguity about what changed.

Contrast — what not to do:

```
# WRONG: Read all 47 files discovered by Glob
# WRONG: Use Write to save the full file after a one-line change
# WRONG: Use Grep with an overly broad pattern like "error" across the whole repo
```

Common Mistakes

1. Reading before searching Jumping to Read without using Grep first forces the agent to process entire files to find a single symbol. Always search before reading.

2. Using Write to modify existing files Write overwrites the complete file. If the file was not read first, the existing content is lost. For any modification to an existing file, use Edit. The Read tool must be called at least once before Edit is valid.

3. Non-unique `old_string` in Edit Edit fails when `old_string` matches more than one location. Provide enough surrounding context — typically 2–3 lines above and below the target — to make the match unique.

4. Glob patterns that are too broad `**/*` on a large monorepo returns thousands of paths. Scope Glob to the relevant subdirectory or file extension to keep results actionable.

5. Reading large files without line ranges A 3,000-line file read in full consumes significant context. Use `offset` and `limit` to read the relevant section. Grep the file first to find the line number.

6. Absolute vs. relative paths All file-path tools require **absolute paths**. Relative paths silently resolve against the process working directory, which may not match the project root. Always confirm the absolute path before passing it.

References

- Claude Code documentation: Built-in tools reference
- Exam domain 2: Tool Selection and Codebase Navigation
- Related handouts: 2.1 (Tool Description Design), 2.4 (Context Window Management)
- See also: `mcp__tool_description_best_practices` — the same principles that govern MCP tool descriptions apply to prompting Claude to select among built-in tools.